File I/O Steps

Basic Rules

- Include the file-handling library before using files.
- Open a file before reading from or writing to it.
- Check that the file opened successfully.
- Use the correct type of stream: one for reading, one for writing, or one for both.
- Read and write using matching data types (what you read must match what was written).
- Stop reading when the read operation fails (don't read past the end).
- Close the file when you are done with it.
- Be careful not to accidentally overwrite important files.

General File I/O Steps

1. Include the header file fstream in the program.
2. Declare file stream variables.
3. Associate the file stream variables with the input/output sources.
4. Open the file
5. Use the file stream variables with >>, <<, or other input/output functions.
6. Close the file.

1.Include the header file fstream in the program.

```
#include <fstream>
```

- file (fstream)
  - **ifstream** - defines new input stream (normally associated with a file).
  - **ofstream** - defines new output stream (normally associated with a file).
  - **fstream** – can be used for both input and output streams

General File I/O Steps

1. Include the header file fstream in the program.

2. ***Declare file stream variables***.

3. Associate the file stream variables with the input/output sources.

4. Open the file

5. Use the file stream variables with >>, <<, or other input/output functions.

6. Close the file.

2. Declare file stream variables.

```
Type variable_name;

int value1;
```

2. Declare file stream variables.

- ifstream in_file;
- ofstream out_file;

General File I/O Steps

1. Include the header file fstream in the program.

2. Declare file stream variables.

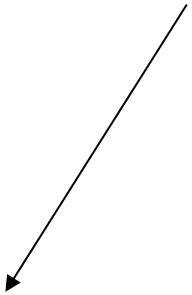
3. ***Associate the file stream variables with the input/output sources.***

4. **Open the file**

5. Use the file stream variables with >>, <<, or other input/output functions.

6. Close the file.

```
in_file.open("babynames.txt");
```



File_name

Open()

- Opening a file associates a file stream variable declared in the program with a physical file at the source, such as a disk.

- In the case of an input file:
  - the file must exist before the open statement executes.
  - If the file does not exist, the open statement fails and the input stream enters the fail state

Open()

- An output file does not have to exist before it is opened;
  - if the output file does not exist, the computer prepares an empty file for output.
  - If the designated output file already exists, by default, the old contents are erased when the file is opened.

Sample

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("babynames.txt");   // open for reading

    if (!file) {                      // check if open failed
        cout << "Could not open babynames.txt\n";
        return 1;
    }

    cout << "babynames.txt opened successfully.\n";
    file.close();                     // optional (will close automatically)
    return 0;
}
```

General File I/O Steps

1. Include the header file fstream in the program.
2. Declare file stream variables.
3. Associate the file stream variables with the input/output sources.
4. Open the file
5. ***Use the file stream variables with >>, <<, or other input/output functions.***
6. Close the file.

Use the file stream variables with >>, <<, or other input/output functions.

```
string word;
in_file >> word;
```

The first character that is not white space becomes the first character in the string word. More characters are added until either another white space character occurs, or the end of the file has been reached. The white space after the word is not removed from the stream.

Note:

White space includes spaces, tab characters, and the newline characters that separate lines

Use the file stream variables with >>, <<, or other input/output functions.

```
string line;
getline(in_file, line);
```

The next input line (without the newline character) is placed into the string line. This could be used to ignore of a number of lines from the input file such as a header!

Use the file stream variables with >>, <<, or other input/output functions.

Similarly

```
out_file << 12.3456789 << " " <<
123456789.0;
```

General File I/O Steps

1. Include the header file fstream in the program.
2. Declare file stream variables.
3. Associate the file stream variables with the input/output sources.
4. Open the file
5. Use the file stream variables with >>, <<, or other input/output functions.
6. ***Close the file.***

Close the file.

When the program ends, all streams that you have opened will be automatically closed. You can also manually close a stream with the close member function:

```
in_file.close();
```

Manual closing is only necessary if you want to use the stream variable again to process another file.

Validation:

After reading data, you should test for success before processing:

```
if (!in_file.fail())
{
Process input.
}
```

Validation:

```
if (in_file >> word)
{
 Process input.
}
```

Writing to Files:

- Files let programs save data permanently.
- For output (writing), C++ uses file output streams.
- Typical use: open file → write data → close file.
- Include: <fstream> for file I/O.
- Use ofstream for writing to files.
- Use fstream for both reading and writing when needed.

Opening a File for Writing:

- Create an ofstream object.
- Provide the filename when opening (e.g., "data.txt").
- Common modes:
- default: create new or overwrite existing file
- app (append): add to the end of an existing file
- binary: write binary data

Checking for Successful Open:

- Always check that the file opened correctly.
- If opening fails, the stream is in a “fail” state.
- Typical causes: wrong path, no permissions, disk errors.
- Use the insertion operator (<<) to send data to the file stream.
- You can write text, numbers, and variables.
- Use newline characters to format lines in the file.
- Close the file when you are done writing.
- Closing ensures all buffered data is written to disk.
- Files are also closed automatically when the stream object is destroyed, but explicit close is good practice.

Sample:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // Create and open a file named babynames.txt for writing
    ofstream outFile("babynames.txt");

    // Check if the file opened successfully
    if (!outFile) {
        cout << "Error: could not open babynames.txt for writing.\n";
        return 1;
    }

    // Write some sample baby names to the file
    outFile << "Emma\n";
    outFile << "Liam\n";
    outFile << "Olivia\n";
    outFile << "Noah\n";

    // Close the file
    outFile.close();

    cout << "Names written to babynames.txt successfully.\n";
    return 0;
}
```